5章

単純な業務ロジックを実装する

業務ロジックはソフトウェアでもっとも重要な部分です。ソフトウェアを開発するそもそもの理由が業務ロジックです。システムのユーザーインターフェースをもっと魅力的にしたり、大規模なデータベースの性能改善に挑戦したりすることもよいでしょう。しかし、そのソフトウェアが業務の役に立たないとしたら、それは高価な技術デモにすぎません。

第2章で見たように、すべての業務領域のソフトウェアを同じように作るわけではありません。業務領域のカテゴリーが異なれば、戦略的な重要性と複雑さが異なります。この章では、業務ロジックがかなり単純な場合に適した実装方法として、トランザクションスクリプトとアクティブレコードを取り上げます。

5.1　トランザクションスクリプト

プレゼンテーション層からのリクエストを処理する手順の中に業務ロジックを埋め込む。

——マーチン・ファウラー＊1

＊1　マーチン・ファウラー著『Patterns of Enterprise Application Architecture』（Addison-Wesley、2002年）（邦訳『エンタープライズアプリケーションアーキテクチャパターン』翔泳社、2005年）

システムが外部に公開するインターフェースは、システムの利用者が実行できる業務処理、つまりトランザクションの集まりと見なせます（図5‐1）。そ

れぞれのトランザクションはシステムが管理するデータを参照したり更新したりします。トランザクションスクリプトは、データを操作する手続きの一部として業務ロジックを記述します。システムの利用者は公開されたインターフェースを使ってその手続きを実行します。この公開された操作の集まりが、さまざまなデータ処理手続きを内部に抱え込んだカプセル化の境界となります。

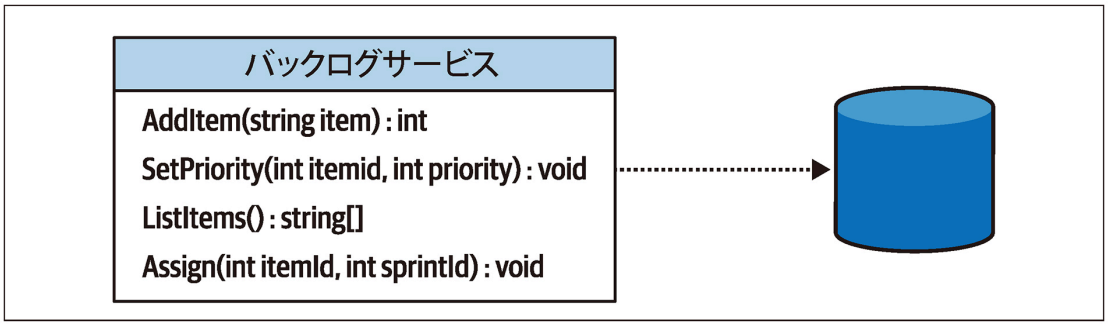

図5-1　トランザクションスクリプトのインターフェース

5.1.1　実装方法

データ操作の手続きを、単純に順次処理式のスクリプトとして記述します。データベースを操作するために薄い抽象化レイヤーをはさむこともできますし、データベースを直接操作することも可能です。

この処理手続きが満たすべき唯一の要件は、トランザクション管理です。処理手続きは、成功または失敗のどちらかで終了します。中途半端な状態で終了してはいけません。トランザクションスクリプトの実行が、もっとも都合の悪いタイミングで失敗しても、データの一貫性を保証する必要があります。それまでに実行した変更をロールバックして開始前の状態に戻す、あるいは、変更した内容を取り消すための補償アクションを実行する必要があります。このトランザクション管理の必要性がトランザクションスクリプトという名前の由来です。

トランザクションスクリプトの例を見てみましょう。以下のコードは、JSON形式のファイルをXML形式に変換するバッチ処理のスクリプトです。

```
DB.StartTransaction();

var job = DB.LoadNextJob();
var json = LoadFile(job.Source);
var xml = ConvertJsonToXml(json);
```

```
WriteFile(job.Destination, xml.ToString();
DB.MarkJobAsCompleted(job);

DB.Commit()
```

5.1.2　それほど単純な話ではない

ドメイン駆動設計講座でトランザクションスクリプトの説明を始めると、受講者はしばしば不満そうな顔をします。「そんな内容は時間の無駄です。もっとレベルの高い技法を教えてください」と言い出す受講者もいます。

そうではありません。トランザクションスクリプトは、この後の章で学ぶ、業務ロジックのより進んだ実装方法の基礎となります。さらに言えば、トランザクションスクリプトは、見た目は単純ですが、使い方をまちがえやすい実装方法です。私はさまざまなデバッグと修正を支援してきましたが、問題の多くは、業務ロジックのトランザクション管理をまちがって実装していたことが原因でした。

実際にあった三つの例を見てみましょう。トランザクションスクリプトの実装をまちがえたことでデータの不整合が起きた例です。

5.1.2.1　トランザクション管理の欠落

データの不整合が発生する単純な例は、一つのトランザクションとして管理しない、複数の更新処理の実行です。以下のコードを見てください。`Users`テーブルのレコードを`UPDATE`し、直後に`VisitsLog`テーブルにレコードを`INSERT`しています。

```
public class LogVisit
{
  ...

  public void Execute(Guid userId, DataTime visitedOn)
  {
    _db.Execute("UPDATE Users SET last_visit=@p1 WHERE user_id=@p2",
      visitedOn, userId);
    _db.Execute(@"INSERT INTO VisitsLog(user_id, visit_date)
          VALUES(@p1, @p2)", userId, visitedOn);
```

```
11   }
12 }
```

`Users`テーブルを`UPDATE`（7行目）した後、9行目のログレコードを書き込む前に障害が起きると、データの不整合が起きます。`Users`テーブルは更新されていますが、それに対応した`VisitsLog`テーブルのレコードを`INSERT`できていない状態になります。ネットワークの切断、データベース処理のタイムアウトやデッドロック、プロセスの異常終了など、さまざまな原因でこういう不整合が発生します。

この場合、以下のコードのように、二つの操作を一つのトランザクションとして実行すれば、データの不整合を防げます。

```
public class LogVisit
{
  ...

  public void Execute(Guid userId, DataTime visitedOn)
  {
    try
    {
      _db.StartTransaction();

      _db.Execute(@"UPDATE Users SET last_visit=@p1
            WHERE user_id=@p2",
            visitedOn, userId);

      _db.Execute(@"INSERT INTO VisitsLog(user_id, visit_date)
            VALUES(@p1, @p2)",
            userId, visitedOn);

      _db.Commit();
    } catch {
      _db.Rollback();
      throw;
    }
  }
}
```

リレーショナルデータベースであれば複数レコードの操作を一つのトランザクションとして扱うことは簡単です。しかし複数レコードの更新を一つのトランザクションとして扱えないデータベースでは、データの一貫性はやっかいな問題になります。あるいは複数のデータベースにまたがった操作を分散トランザクションとして管理できない場合も同様です。後者の例を見てみましょう。

5.1.2.2　分散トランザクション

最近の分散システムでは、データベースのデータを変更した時に、メッセージをメッセージ通信基盤に送出して、変更の発生を他のコンポーネントに通知することが普通に行われています。さきほどの例でいえば、サイトへの訪問を `VisitLog` テーブルに記録するかわりに、以下のようなやり方で、メッセージをメッセージ通信基盤に送出します。

```
01 public class LogVisit
02 {
03   ...
04
05   public void Execute(Guid userId, DataTime visitedOn)
06   {
07     _db.Execute("UPDATE Users SET last_visit=@p1 WHERE user_id=@p2",
08           visitedOn,userId);
09     _messageBus.Publish("VISITS_TOPIC",
10               new { UserId = userId, VisitDate = visitedOn });
11   }
12 }
```

前の例と同じように、7行目と9行目の間で何らかの障害が発生すればデータの一貫性が崩れます。たとえば、 `Users` テーブルの変更は成功しているが、メッセージ通信基盤の障害のため、他のコンポーネントへの通知が失敗している状況です。

残念ながら、この問題の解決方法は、先ほどの例のように簡単ではありません。複数データベースにデータを書き込む分散トランザクションは複雑な仕組みです。スケールすることが難しく、エラーも起きやすくなります。したがって、多くの場合、分散トランザクションは使いません。**第8章**で、複数データベースにデータを書き込む方法として、コマンド・クエリ責任分離（CQRS）を学びます。また、**第9章**では、他のコンポーネントに変更を確実に通知する

ために、変更の発生を別のデータベースに記録してからメッセージを発行する方法として、送信箱（Outbox）を紹介します。

トランザクション管理のまちがった実装として、もっと複雑な例を見てみましょう。

5.1.2.3　暗黙的な分散トランザクション

一見単純そうに見える、以下のメソッドを考えてみましょう。

```
public class LogVisit
{
  ...

  public void Execute(Guid userId)
  {
    _db.Execute("UPDATE Users SET visits=visits+1 WHERE user_id=@p1",
          userId);
  }
}
```

さきほどまでの例では、最新の訪問日を記録していました。しかしこのメソッドではユーザーごとの訪問回数（`visits`）を更新しています。このメソッドを呼ぶたびにユーザーの訪問回数が1回増えます。このメソッドは一つの値を更新しているだけです。更新の対象は特定データベースの特定テーブルだけです。しかし、実際には、このメソッドはデータの不整合につながる分散トランザクションです。

この例がなぜ分散トランザクションになるのでしょうか。それは、この`LogVisit`クラスは、データベースと通信し、かつ、メソッドを呼び出した外部のプロセス（クライアント）と通信を行っているからです。図5‐2を見てください。

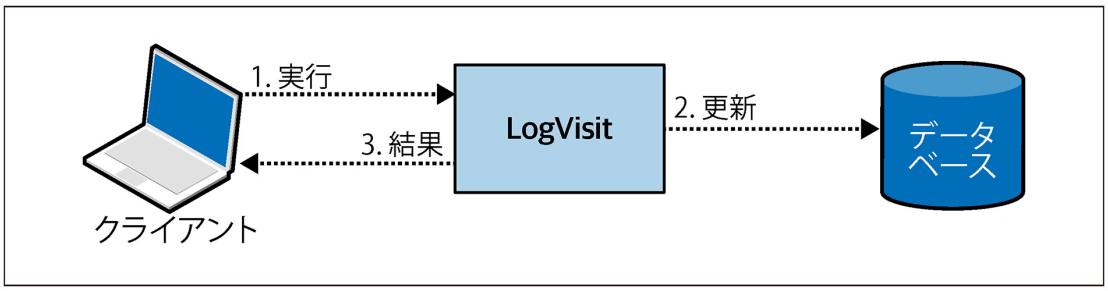

図5-2　LogVisit操作はデータを更新し、その結果を呼び出し元に通知する

`Execute`メソッドは`void`型のメソッドです。つまり実行結果としてデータを返しません。しかし、このメソッドは、操作の成功または失敗をクライアントに通知します。つまり、失敗した場合、呼び出し元は例外を受け取ります。ではメソッド実行は成功したが、結果を返す通信に失敗した場合はどうなるでしょうか。たとえば次のような状況です。

- RESTサービスで`LogVisit`の処理の途中で通信が切断された、あるいは、
- `LogVisit`の呼び出し元は同じプロセスで実行されているが、呼び出し元が`LogVisit`の処理完了を確認する前にプロセスが停止した

どちらの場合も呼び出し元は`Execute`メソッドが失敗したと判断します。そして`LogVisit`を再実行し、訪問回数は1回増えます。結果として訪問回数は1回ではなく2回になってしまいます。前の二つの例と同じように、このコードもトランザクションスクリプトを正しく実装できていません。気がつかないうちにデータの不整合が起きてしまいます。

前の例と同じように、この場合も問題を解決するのは簡単ではありません。どう解決するかは、対象業務の特性と業務側の要求に依存します。この例に限れば、一つの方法して、操作を「べき等」（idempotent）にすることでデータの一貫性を実現できます。べき等とは、同じ操作を繰り返しても、常に同じ結果になる性質を指します。

たとえば、呼び出し元が訪問回数を渡すようにする、という方法があります。訪問回数を渡すために、呼び出し元は、以下の手順で操作を行います。

1. 現在の訪問回数を取得する
2. 訪問回数を1回増やす
3. 増やした結果の訪問回数をメソッドの引数として渡す

この方法であれば同じ操作を複数回実行しても訪問回数は正しく記録されます。たとえば次のやり方です。

```
public class LogVisit
{
  ...

  public void Execute(Guid userId, long visits)
  {
```

```
      _db.Execute("UPDATE Users SET visits = @p1 WHERE user_id=@p2",
            visits, userId);
    }
  }
```

この問題を解決するもう一つのやり方は、楽観的な排他制御を使う方法です。つまり`LogVisit`を呼び出す前に現在の訪問回数を取得し、その値を`LogVisit`に渡します。呼び出された`LogVisit`側は、渡された訪問回数と現在の訪問回数が同じ場合に限り、訪問回数を更新します。たとえば次のやり方です。

```
  public class LogVisit
  {
  ...

    public void Execute(Guid userId, long expectedVisits)
     {
      _db.Execute(@"UPDATE Users SET visits=visits+1
             WHERE user_id=@p1 and visits = @p2",
             userId, expectedVisits);
    }
  }
```

引数に渡す現在の訪問回数が同じまま`LogVisit`の実行を繰り返しても、`WHERE ...visits = @p2`が成立しないので、データベースの訪問回数は更新されません。

5.1.3　トランザクションスクリプトをいつ使うか

トランザクションスクリプトが向いているのは、業務ロジックが手続き的なデータ操作だけ、という単純な問題領域です。たとえば、データのETL（抽出、変換、書き出し）操作です。データソースから必要なデータを抽出し、データ形式の変換を行い、その結果を出力先に書き込むといった**図5‐3**のような処理です。

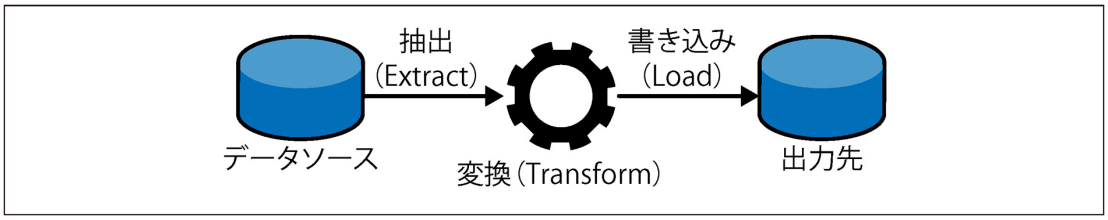

図5-3　抽出、変換、書き込みのデータの流れ

トランザクションスクリプトは補完的な業務領域に向いています。この領域は、そもそもの定義として、業務ロジックは単純です。また外部システムと連係するアダプターとしても使えます。たとえば、一般的な業務領域に導入した外部サービスとの連係や、区切られた文脈をつなぐ時のモデル変換装置の一部として、トランザクションスクリプトは役に立つでしょう（この点は**第9章**で検討します）。

トランザクションスクリプトのよい点は単純さです。抽象化は最小限です。実行時のオーバーヘッドは小さく、業務ロジックは簡単に理解できます。しかしそれは欠点でもあります。業務ロジックが複雑になってくると、あちこちのトランザクションスクリプトで同じ業務ロジックを重複して記述することになります。その結果、業務ロジックの一貫性が保てなくなり、不具合の原因になります。ですから、トランザクションスクリプトを中核の業務領域に使ってはいけません。トランザクションスクリプトは、中核の業務領域の複雑な業務ロジックに対応できません。

トランザクションスクリプトのこの単純さは、あまり評判がよくはありません。トランザクションスクリプトはアンチパターンとされることさえあります。確かに複雑な業務ロジックをトランザクションスクリプトで実装すれば、あっという間に変更がやっかいで危険な大きな泥団子になってしまうでしょう。しかし、トランザクションスクリプトは広く採用されている実装方法であることを忘れてはいけません。この章とその後の章で検討する実装方法は、何らかの形でトランザクションスクリプトが基礎になっています。

5.2　アクティブレコード

データベースのテーブルまたはビューの1行を表すオブジェクト。データベース操作をカプセル化し、データに関連する業務ロジックを持つ。

——マーチン・ファウラー**＊2**

＊2　マーチン・ファウラー著『エンタープライズアプリケーションアーキテクチャパターン』（翔泳社、2005年）

トランザクションスクリプトと同じように、アクティブレコードも単純な業務ロジックの実装方法です。しかし、アクティブレコードは、複雑なデータ構造を操作できます。たとえば、フラットなデータ構造ではなく、図5‐4のような入り組んだ構造のオブジェクトを扱うことが可能です。

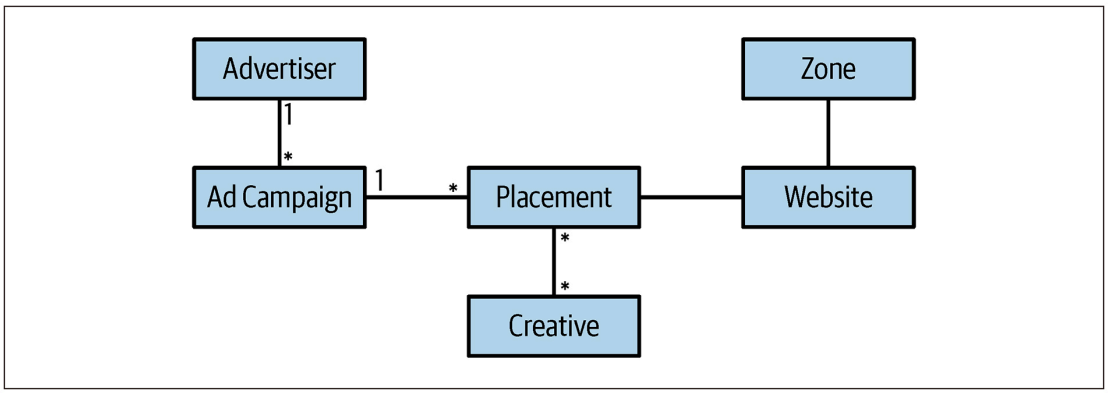

図5-4　1対多や多対多の構造を持つ複雑なデータモデル

このようなデータ構造の操作をトランザクションスクリプトで記述すると、似たようなコードの繰り返しになります。データをメモリ上のオブジェクトに変換するコードを、あちこちに重複して記述することになるでしょう。

5.2.1　実装方法

それを避けるために、データ構造が複雑な場合、アクティブレコードと呼ばれる専用のオブジェクトを使います。このオブジェクトは、データ構造だけでなく、レコードの作成、読み取り、更新、削除をするための、いわゆるCRUD操作のメソッドを実装します。結果として、アクティブレコードオブジェクトは、オブジェクト-リレーショナルマッピング（ORM：Object-Relational Mapping）などデータベース操作用のフレームワークと密に結合します。アクティブレコードという名称は、データ構造を表現するオブジェクト自身が能動的（アクティブ）にデータを操作することに由来します。

アクティブレコードを使う場合も、業務ロジックはトランザクションスクリプトとして記述します。この場合、トランザクションスクリプトは、データベースを直接操作するのではなく、アクティブレコードオブジェクトを操作しま

す。操作の結果は、以下のように、アトミックなトランザクションとして、完了か失敗かのどちらかになります。

```
public class CreateUser
{
  ...

  public void Execute(UserDetails userDetails)
  {
    try
    {
      _db.StartTransaction();

      var user = new User(); //アクティブレコードオブジェクトの生成
      user.Name = userDetails.Name;
      user.Email = userDetails.Email;
      user.Save();

      _db.Commit();
    } catch {
      _db.Rollback();
      throw;
    }
  }
}
```

アクティブレコードは、メモリ上のオブジェクトとデータベースのテーブル構造との複雑なマッピングをカプセル化します。アクティブレコードはオブジェクトの永続化だけでなく、業務ロジックを含むことも可能です。たとえば、レコード作成時のデータの妥当性検査やオブジェクトのデータを使った業務上の計算判断です。しかしアクティブレコードの典型的な使い方は、データ構造と振る舞い（業務ロジック）の分離です。たいていの場合、アクティブレコードがgetter/setterメソッドを公開し、外部の手続きがアクティブレコードのフィールドを変更します。

5.2.2　アクティブレコードをいつ使うか

アクティブレコードも本質的にはトランザクションスクリプトです。アクティブレコードの主たる目的はデータベース操作の最適化です。ですのでアクテ

ィブレコードの用途は、CRUD操作と入力値の妥当性検査のような、比較的簡単な業務ロジックに限定されます。

つまりアクティブレコードを使うのは、トランザクションスクリプトと同じく、以下の場合です。

- 補完的な業務領域
- 一般的な業務領域用の外部サービスとの連係
- 区切られた文脈どうしを連係する時のモデルの変換

アクティブレコードは、複雑なデータ構造をデータベースのテーブル構造にマッピングするために使います。

アクティブレコードは「貧血ドメインモデル」と呼ばれるアンチパターンとして知られています。別の言い方をすると、ドメインモデルの設計として不適切とされています。しかし、私は、「貧血」とか「アンチパターン」という否定的な扱い方は避けたいと思います。アクティブレコードは一つの手段です。どんな手段でもそうですが、使い方によっては役に立つはずです。もちろんどんな手段も、まちがった目的で使えば、問題のほうが大きくなるでしょう。業務ロジックが単純な場合であれば、アクティブレコードを使うことに問題はありません。もっと言えば、業務ロジックが単純な場合、手の込んだ設計で実装するほうが、不必要な複雑さを持ち込むことになるでしょう。次の**第6章**で、ドメインモデルとは何か、そしてドメインモデルとアクティブレコードの違いについて学びます。

ここで説明しているアクティブレコードは設計パターンです。ORMフレームワークのActive Recordではありません。設計パターンとしてのアクティブレコードは、マーチン・ファウラーが『エンタープライズアプリケーションアーキテクチャパターン』（翔泳社）で紹介しました。この設計パターンの実装方法の一つとして後から生まれたのがフレームワークとしてのActive Recordです。この本で説明しているのはアクティブレコードという、設計の考え方です。特定の実装方法を取り上げているわけではありません。

5.3　現実的に考える

業務データは重要です。コードを書く時にデータの一貫性を保証しなければなりません。しかし、現実的な選択が必要な場合もあります。

特に大規模なデータ処理が必要な場合、データの一貫性に対して、柔軟なアプローチを選択することもあります。百万件のレコードに対してたった1件の正しくないレコードを検出した場合、業務全体を停止すべきでしょうか。その1件の不正レコードが事業に深刻な不利益をもたらすでしょうか。たとえばIoTデバイスから毎日数十億件のイベントデータを取り込むことを考えてみてください。0.001%のイベントが重複したり失われたりすることが、はたして重大事件でしょうか。

どんな状況にも適用できる汎用ルールはありません。どんな事業領域を対象にソフトウェアを開発しているかによって設計判断の基準は異なります。可能な範囲で特殊なケースを切り捨てることも選択肢の一つです。特殊なケースを切り捨てることによるリスクと事業への影響を評価して、設計判断をしてください。

5.4　まとめ

この章では単純な業務ロジックの実装方法として、次の二つを説明しました。

トランザクションスクリプト

システムの操作を単純でわかりやすい手続き的なスクリプトとして記述します。それぞれの手続きはトランザクションを管理し、データ操作が完全に成功するか、それとも完全に失敗するかのどちらかであることを保証します。トランザクションスクリプトの主な用途は補完的な業務領域です。業務ロジックが単純で、ETL的な操作に適しています。

アクティブレコード

業務ロジックは単純だがデータ構造が複雑な場合、複雑なデータ構造をアクティブレコードオブジェクトとして実装することが可能です。アクティブレコードオブジェクトは単純なCRUD操作のメソッドを持つデータ構造です。

この章で検討した二つの実装方法は、業務のロジックがかなり単純な場合の選択肢です。次の**第6章**では、複雑な業務ロジックに焦点を合わせ、ドメインモデルを使って業務ロジックの複雑さにどう立ち向かうかを検討します。

5.5　演習問題

1. 検討してきた業務ロジックの実装方法で、中核の業務領域に適した方法はどれでしょう？
   A. トランザクションスクリプト
   B. アクティブレコード
   C. どちらも中核の業務領域には向かない
   D. どちらも中核の業務領域に適している
2. 以下のコードで考えてみてください。

```
public void CreateTicket(TicketData data)
{
  var agent = FindLeastBusyAgent();

  agent.ActiveTickets = agent.ActiveTickets + 1;
  agent.Save();

  var ticket = new Ticket();
  ticket.Id = Guid.New();
  ticket.Data = data;
  ticket.AssignedAgent = agent;
  ticket.Save();
```

```
        _alerts.Send(agent, "You have a new ticket!");
    }
```

トランザクション管理をしていない場合、どのようなデータ不整合の可能性があるでしょうか？

1. 新しいチケットを受け付けた時、チケットを割り当てられた担当者の対応中のチケット数が1以上増えてしまう
2. 担当者の対応中のチケット数が増えるが、実際にはチケットが割り当てられていない
3. 担当者にチケットを割り当てたが、その通知が担当者に届かない
4. 上記の三つとも起こり得る

3. 前述のコードでは、データの不整合が起きる可能性がもう一つあります。それは何かわかりますか？
4. 「この本について」で紹介したウルフデスク社のサービスで、システムのどの部分がトランザクションスクリプトあるいはアクティブレコードで実装できそうですか？